

第三章

(第 3、4、5、7、8、10 题)

3. 对于以下 AT&T 格式汇编指令，根据操作数的长度确定对应指令助记符中的长度后缀，并说明每个操作数的寻址方式。

- (1) `mov 8(%ebp, %ebx, 4), %ax`
- (2) `mov %al, 12(%ebp)`
- (3) `add (, %ebx, 4), %ebx`
- (4) `or (%ebx), %dh`
- (5) `push %rcx`
- (6) `mov $0xFFF0, %eax`
- (7) `test %rax, %rax`
- (8) `lea 8(%ebx, %esi), %eax`

参考答案：

- | | | |
|------------|----------------|----------|
| (1) 后缀: w, | 源: 基址+比例变址+偏移, | 目: 寄存器 |
| (2) 后缀: b, | 源: 寄存器, | 目: 基址+偏移 |
| (3) 后缀: l, | 源: 比例变址, | 目: 寄存器 |
| (4) 后缀: b, | 源: 基址, | 目: 寄存器 |
| (5) 后缀: q, | 源: 立即数, | 目: 栈 |
| (6) 后缀: l, | 源: 立即数, | 目: 寄存器 |
| (7) 后缀: q, | 源: 寄存器, | 目: 寄存器 |
| (8) 后缀: l, | 源: 基址+变址+偏移, | 目: 寄存器 |

4. 使用汇编器处理以下 IA-32/x86-64 中 AT&T 格式代码时都会产生错误，请说明每一行存在什么错误。

- (1) `movl 0xFF, (%eax)`
- (2) `movb %ax, 12(%ebp)`
- (3) `addl %ecx, $0xF0`
- (4) `orw $0xFFFF0, (%ebx)`
- (5) `addb $0xF8, (%dl)`
- (6) `movl %bx, %eax`
- (7) `andl %esi, %esx`
- (8) `movq 8(%ebp, , 4), %rax`
- (9) `leaq 20(%rdi, %rsi), %eax`

参考答案：

- (1) 源操作数是立即数 0xFF，需在前面加 '\$'
- (2) 源操作数是 16 位，而长度后缀是字节 'b'，不一致
- (3) 目的操作数不能是立即数寻址
- (4) 操作数位数超过 16 位，而长度后缀为 16 位的 'w'
- (5) 不能用 8 位寄存器作为目的操作数地址所在寄存器
- (6) 源操作数寄存器与目操作数寄存器长度不一致
- (7) 不存在 ESX 寄存器
- (8) 源操作数地址中缺少变址寄存器，而且从长度后缀 q 和目的寄存器 RAX 可知，这是 x86-64 中的指令，因此，基址寄存器应该是 RBP，而不是 EBP

(9) 目的寄存器应是 64 位寄存器

5. 假设变量 x 和 ptr 的类型声明如下：

```
src_type x;
```

```
dst_type *ptr;
```

这里，src_type 和 dst_type 是用 typedef 声明的数据类型。有以下一个 C 语言赋值语句：

```
*ptr=(dst_type) x;
```

若 x 存储在寄存器 EAX 或 AX 或 AL 中，ptr 存储在寄存器 EDX 中，则对于表 3.10 中给出的 src_type 和 dst_type 的类型组合，写出实现上述赋值语句的 IA-32 机器级代码。要求用 AT&T 格式汇编指令表示机器级代码。

表 3.10 题 5 用表

src_type	dst_type	机器级表示
char	int	
int	char	
int	unsigned	
short	int	
unsigned char	unsigned	
char	unsigned	
int	int	

参考答案：

表 3.10 题 5 用表

src_type	dst_type	机器级表示
char	int	movsbl %al, %eax movl %eax, (%edx)
int	char	movb %al, (%edx)
int	unsigned	movl %eax, (%edx)
short	int	movswl %ax, %eax movl %eax, (%edx)
unsigned char	unsigned	movzbl %al, %eax movl %eax, (%edx)
char	unsigned	movsbl %al, %eax movl %eax, (%edx)
int	int	movl %eax, (%edx)

7. 假设在 IA-32 系统中以下地址以及寄存器中存放的机器数如表 3.11 所示。

表 3.11 题 7 用表

地址	机器数	寄存器	机器数
0x0804 9300	0xffff fff0	EAX	0x0804 9300
0x0804 9400	0x8000 0008	EBX	0x0000 0100
0x0804 9384	0x80f7 ff00	ECX	0x0000 0010
0x0804 9380	0x908f 12a8	EDX	0x0000 0080

分别说明执行以下指令后，哪些存储单元地址或寄存器中的内容会发生改变？改变后的内容是什么？

么？条件标志 OF、SF、ZF 和 CF 会发生什么改变？

- (1) `addl (%eax), %edx`
- (2) `subl (%eax, %ebx), %ecx`
- (3) `orw 4(%eax, %ecx, 8), %bx`
- (4) `testb $0x80, %dl`
- (5) `imull $32, (%eax, %edx), %ecx`
- (6) `mulw %bx`
- (7) `decw %cx`

参考答案：

- (1) 指令功能为： $R[edx] \leftarrow R[edx] + M[R[edx]] = 0x00000080 + M[0x8049300]$ ，寄存器 EDX 中内容改变。改变后的内容为以下运算的结果：00000080H+FFFFFFFFH

$$\begin{array}{r} 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1000\ 0000 \\ +\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 0000 \\ \hline 1\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0111\ 0000 \end{array}$$

因此，EDX 中的内容改变为 0x00000070。根据书中表 3.8 可知，加法指令会影响 OF、SF、ZF 和 CF 标志。OF=0，ZF=0，SF=0，CF=1。

- (2) 指令功能为： $R[ecx] \leftarrow R[ecx] - M[R[ecx] + R[ebx]] = 0x00000010 + M[0x8049400]$ ，寄存器 ECX 中内容改变。改变后的内容为以下运算的结果：00000010H-80000008H

$$\begin{array}{r} 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001\ 0000 \\ +\ 0111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1000 \\ \hline 0\ 1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1000 \end{array}$$

因此，ECX 中的内容改为 0x80000008。根据书中表 3.8 可知，减法指令会影响 OF、SF、ZF 和 CF 标志。OF=1，ZF=0，SF=1，CF=1 $\oplus$ 0=1。

- (3) 指令功能为： $R[bx] \leftarrow R[bx] \text{ or } M[R[ecx] + R[ebx] * 8 + 4]$ ，寄存器 BX 中内容改变。改变后的内容为以下运算的结果：0x0100 or M[0x8049384]=0100H or FF00H

$$\begin{array}{r} 0000\ 0001\ 0000\ 0000 \\ \text{or } 1111\ 1111\ 0000\ 0000 \\ \hline 1111\ 1111\ 0000\ 0000 \end{array}$$

因此，BX 中的内容改为 0xFF00。由 3.3.3 节可知，OR 指令执行后 OF=CF=0；因为结果不为 0，故 ZF=0；因为最高位为 1，故 SF=1。

- (4) `test` 指令不改变任何通用寄存器，但根据以下“与”操作改变标志： $R[dl] \text{ and } 0x80$

$$\begin{array}{r} 1000\ 0000 \\ \text{and } 1000\ 0000 \\ \hline 1000\ 0000 \end{array}$$

由 3.3.3 节可知，TEST 指令执行后 OF=CF=0；因为结果不为 0，故 ZF=0；因为最高位为 1，故 SF=1。

- (5) 指令功能为： $R[ecx] \leftarrow M[R[ecx] + R[edx]] * 32$ ，即 ECX 中的内容改变为以下运算的结果：
 $M[0x8049380] * 32 = 0x908f12a8 * 32$ (带符号整数乘)，只要将 0x908f12a8 左移 5 位即可得到结果。
1001 0000 1000 1111 0001 0010 1010 1000 << 5
= 0001 0001 1110 0010 0101 0101 0000 0000

因此，指令执行后，ECX 中的内容改变为 0x11e25500。显然，这个结果是溢出的。根据书中表 3.8 可知，乘法指令会影响标志位 OF 和 CF，溢出时 OF=CF=1。

- (6) 指令功能为：R[dx-ax]←R[ax]*R[bx]（无符号整数乘，结果取 32 位乘积），即 DX 和 AX 中的内容改变为以下运算的结果：0x9300*0x0100，只要将 0x9300 左移 8 位（高位补 0）即可得到结果。

1001 0011 0000 0000<<8=0000 0000 1001 0011 0000 0000 0000 0000

因此，指令执行后，DX 中的内容变为 0x0093，AX 中内容变为全 0。因为 DX 中的高 16 位不为 0，所以取低 16 位作为乘积时结果溢出。根据书中表 3.8 可知，乘法指令会影响标志位 OF 和 CF，溢出时 OF=CF=1。

- (7) 指令功能为：R[cx]←R[cx]-1，即 CX 寄存器的内容减 1。

```

      0000 0000 0001 0000
+   1111 1111 1111 1111
-----
1 0000 0000 0000 1111

```

因此，指令执行后 CX 中的内容从 0x0010 变为 0x000F。由书中表 3.8 可知，DEC 指令会影响 OF、ZF、SF，根据上述运算结果，得到 OF=0，ZF=0，SF=0。

8. 已知 IA-32 采用小端方式，根据给出的 IA-32 机器代码的反汇编结果（部分信息用 x...x 表示）回答问题。

- (1) 已知 je 指令的操作码为 0111 0100，je 指令的跳转目标地址是什么？call 指令中的跳转目标地址 0x80483b1 是如何反汇编出来的？

```

804838c: 74 08          je      xxxxxxxx
804838e: e8 1e 00 00 00 call    80483b1<test>

```

- (2) 已知 jb 指令的操作码为 0111 0010，jb 指令的跳转目标地址是什么？movl 指令中的目的地址是如何反汇编出来的？

```

8048390: 72 f6          jb      xxxxxxxx
8048392: c6 05 00 a8 04 08 01 movl    $0x1, 0x804a800
8048399: 00 00 00

```

- (3) 已知 jle 指令的操作码为 0111 1110，jle 和 mov 指令的地址各是什么？

```

xxxxxxx: 7e 16          jle     80492e0
xxxxxxx: 89 d0          mov     %edx, %eax

```

- (4) 已知 jmp 指令的跳转目标地址采用相对寻址方式，jmp 指令操作码为 1110 1001，其跳转目标地址是什么？

```

8048296: e9 00 ff ff ff jmp     xxxxxxxx
804829b: 29 c2          sub     %eax, %edx

```

参考答案：

根据书中 3.3.4 节得知，条件跳转指令都采用相对跳转方式在段内直接跳转，即条件跳转指令的跳转目标地址为：(PC)+偏移量。此处 PC 的内容是条件跳转指令的下一条指令的地址。

- (1) 因为 je 指令的操作码为 0111 0100，所以机器代码 7408H 中的 08H 是偏移量，故跳转目标地址为：0x804838c+2+0x8=0x8048396。

call 指令中的跳转目标地址 0x80483b1=0x804838e+5+0x1e，由此，可以看出，call 指令机器代码中后面的 4 个字节是偏移量，因 IA-32 采用小端方式，故偏移量为 0000001EH。call 指令机器代码共占 5 个字节，因此，下条指令的地址为当前指令地址 0x804838e 加 5。

- (2) jb 指令中 F6H 是偏移量，故其跳转目标地址为：0x8048390+2+0xf6=0x8048388。

movl 指令的机器代码有 10 个字节，前两个字节是操作码等，后面 8 个字节为两个立即数，因为是小端方式，所以，第一个立即数为 0804A800H，即汇编指令中的目的地址 0x804a800，最后 4 个字节为立即数 00000001H，即汇编指令中的常数 0x1。

(3) jle 指令中的 7EH 为操作码，16H 为偏移量，其汇编形式中的 0x80492e0 是跳转目的地址，因此，假定后面的 mov 指令的地址为 x，则 x 满足以下公式： $0x80492e0 = x + 0x16$ ，故 $x = 0x80492e0 - 0x16 = 0x80492ca$ ，因而前一条的 jle 指令的地址为 $0x80492ca - 2 = 0x80492c8$ 。

(4) jmp 指令中的 E9H 为操作码，后面 4 个字节为偏移量，因为是小端方式，故偏移量为 FFFFFFF0H，即 -100H=-256。后面的 sub 指令的地址为 0x804829b，故 jmp 指令的跳转目标地址为 $0x804829b + 0xffffffff = 0x804829b - 0x100 = 0x804819b$ 。

10. 假设 x86-64 系统某程序中变量 x 和 ptr 的类型声明如下：

```
src_type x;
```

```
dst_type *ptr;
```

这里，src_type 和 dst_type 是用 typedef 声明的数据类型。有以下几个 C 语言赋值语句：

```
*ptr=(dst_type) x;
```

若 x 存储在寄存器 RAX 或 EAX 或 AX 或 AL 中，ptr 存储在寄存器 RDX 中，则对于表 3.12 给出的 src_type 和 dst_type 的类型组合，写出实现上述赋值语句对应的汇编指令。

表 3.12 题 10 表

src_type	dst_type	汇编指令（AT&T 格式）
char	long	
int	char	
int	unsigned long	
short	int	
unsigned char	unsigned	
char	unsigned long	
unsigned long	int	

参考答案：

表 3.12 题 10 表

src_type	dst_type	汇编指令（AT&T 格式）
char	long	movsbq %al, %rax movq %rax, (%rdx)
int	char	movb %al, (%rdx)
int	unsigned long	movslq %eax, %rax movq %rax, (%rdx)
short	int	movswl %ax, %eax movl %eax, (%rdx)
unsigned char	unsigned	movzbl %al, %eax movl %eax, (%rdx)
char	unsigned long	movsbq %al, %rax movq %rax, (%rdx)
unsigned long	int	movl %eax, (%rdx)